

Pill-터링 (Team_3)

AI 기반 경구용 알약 이미지 인식 및 분류 시스템

[Entry-Level] Object Detection Project

Kaggle mAP
0.99451

채택 모델
YOLOv8s

팀 구성

이름	역할
유연정 (팀장)	PM / 데이터 전처리 / 모델 설계 및 실험
김현숙	데이터 전처리 / 모델 설계 및 실험
이현진	모델 설계 및 실험 / PPT 작성
송우현	모델 설계 및 실험
은광성	데이터 전처리 / 모델 실험

제출일: 2026. 04. 01.

AI 부트캠프 초급 프로젝트

1. 프로젝트 개요

본 프로젝트는 경구용 알약 이미지를 인식하고 분류하는 객체 탐지(Object Detection) 모델을 개발하는 것을 목표로 한다. 캐글(Kaggle) 대회 형식으로 진행되었으며, mAP(Mean Average Precision)를 핵심 평가 지표로 사용하였다.

데이터셋은 총 1,489장의 학습 이미지와 73개의 알약 카테고리, 4,526개의 annotation으로 구성되어 있다. 하나의 이미지에 여러 알약이 존재하는 경우가 많으며, 카테고리 간 데이터 불균형이 뚜렷하게 나타나는 특징을 가진다. (annotation수가 img보다 더 많이 존재한다는 사실은 하나의 이미지 안에 여러 알약의 정보가 담겨있다는 뜻으로, 이러한 특징은 그림 2와 3 에서처럼, 각 이미지 내에 포함된 알약 개수 분포를 통해 확인해 볼 수 있다.)

```
=== 이미지당 알약 개수 통계 ===
count      1489.000000
mean        3.039624
std         0.837889
min         1.000000
25%         3.000000
50%         3.000000
75%         4.000000
max         4.000000
dtype: float64

0개 (annotation 없음): 0장
1개 : 64장
2개 : 302장
3개 : 634장
4개 : 489장
5개 이상 : 0장
```

그림 1. train dataset 통계

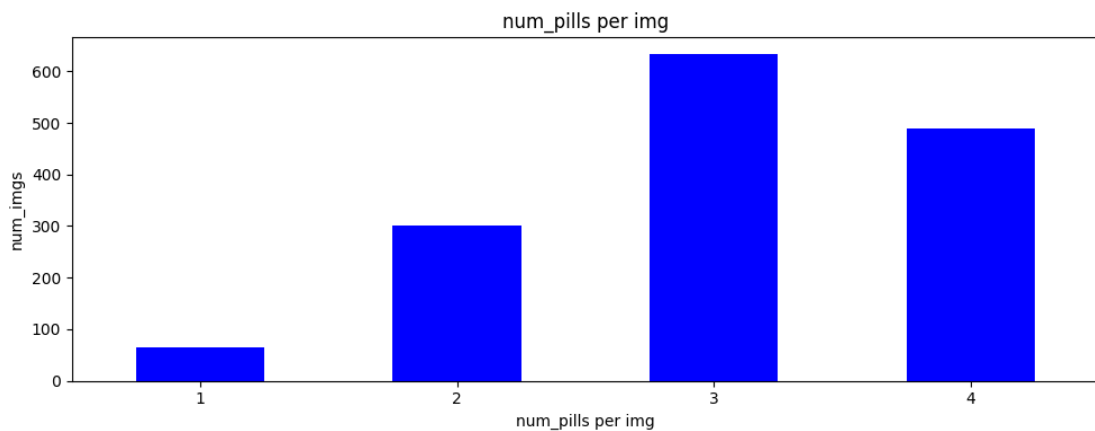


그림 2. 하나의 이미지당 annotation이 정의된 알약 수

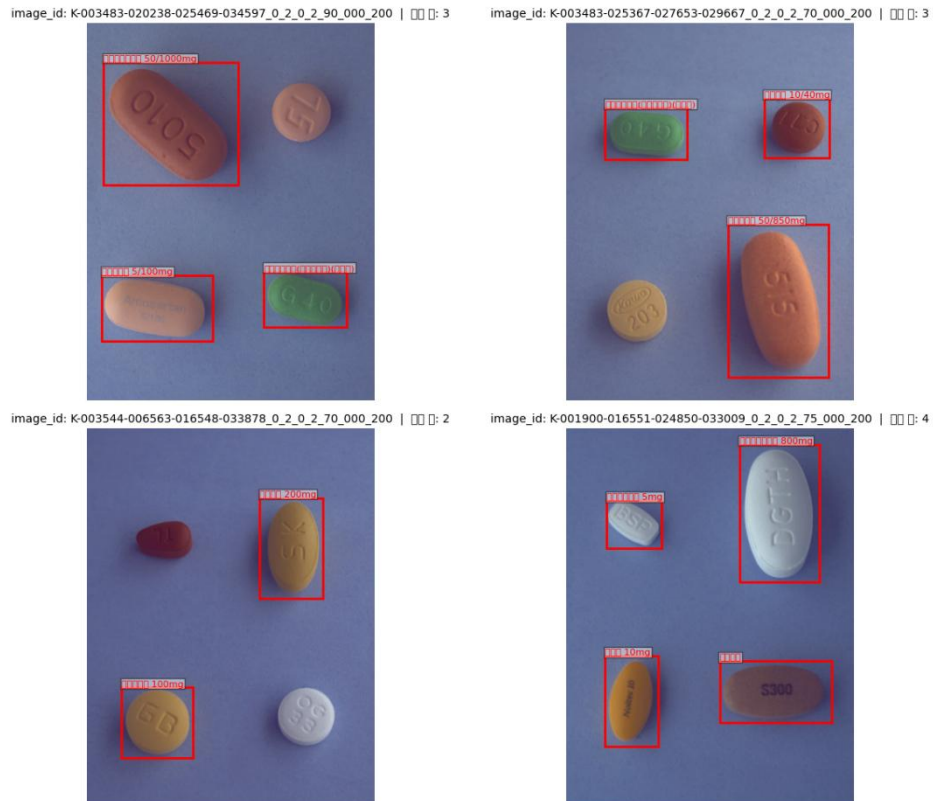


그림 3. 시각화(random)

이 과정에서 발견할 수 있는 사실은, 하나의 이미지 안에 여러 개의 알약이 존재함에도 불구하고 일부 알약 객체에 대한 annotation이 부재한 경우가 존재한다는 점이다.

```

=== category별 annotation 수 ===
1899      180
2482      172
3350      240
3482      514
3543       99
...
35205     113
36636     108
38161      27
41767      23
44198      30
Name: count, Length: 73, dtype: int64

최대: 514개 | 최소: 7개
평균: 62.0개

```

그림 4. cat_id에 따른 annotation수

각 알약 객체에 따른 annotation수의 편차가 존재한다는 사실을 그림 4을 통해 확인할 수 있다. 이 수치를 그림 5의 그래프로 살펴보면, 그 차이를 명확하게 확인할 수 있는데, 가장 많이 등장하는 알약 클래스의 annotation 수는 514개인 반면, 가장 적은 경우는 7개로 나타난다.

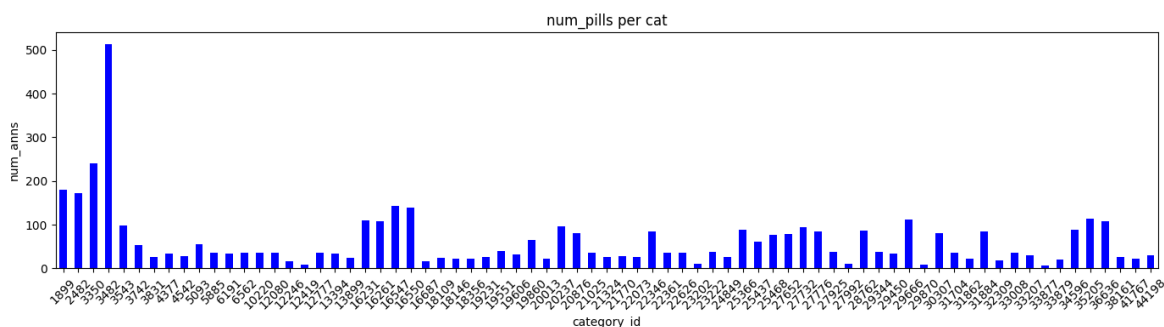


그림 5. cat_id 별 알약 수

결과적으로, 이 과정에서 이미지와 annotation 구조에서 두 가지 주요 문제점을 확인할 수 있었다.

첫 번째는 알약 클래스 간 데이터 분포의 불균형class imbalance이며, 두 번째는 일부 이미지에서 알약 객체가 존재함에도 해당 객체에 대한 annotation이 부재한다는 점이다. annotation 누락 문제의 경우, 추가적인 데이터를 생성하거나 보완하기 어렵기 때문에, 이어지는 제안점에서는 데이터 불균형 문제를 완화하는 방법으로 전처리 전략을 설계하고 실험을 진행하였다.

프로젝트는 데이터 분석 및 전처리 → 모델 탐색 및 비교 → 반복 실험 및 하이퍼파라미터 튜닝 → 최종 모델 선정의 흐름으로 진행되었으며, 팀원 5명이 역할을 분담하여 협업하였다.

2. 최종 결과

지표	결과
Kaggle mAP	0.99451
최종 순위	1등
최종 채택 모델	YOLOv8s
초기 점수 (YOLOv8s 첫 제출)	0.989
중간 점수	0.993

3. 실험 과정 및 사용 모델

3.1 Phase 1 — Baseline: Faster R-CNN 전처리 및 실험

프로젝트 시작 시 제공된 Faster R-CNN(ResNet50 FPN) 베이스라인 코드를 기반으로 데이터 분석 및 전처리 실험을 우선 진행하였다. 초기 5 epoch 기준 mAP@0.5는 0.426, mAP@0.5:0.95는 0.398이었다.

데이터 구조 분석 결과, 카테고리당 annotation 수 편차가 최대(514개) 대 최소(7개)로 매우 크게 나타났다. 이에 따라 다음 세 가지 전처리 방향을 설계하고 실험하였다.

- Outlier Handling: annotation 수 400 개 이상인 클래스를 300 개로 제한
 - A. 앞서 그림 5 의 분석 결과를 바탕으로 데이터 불균형을 각 구간별로 정리한 결과는 그림 6 와 같다. annotation 수가 400 개 이상인 클래스는 1 개, 1~100 구간에는 62 개, 101~200 구간에는 9 개, 201~300 구간에는 1 개의 클래스가 포함되어 있는 것으로 나타났다. 이에 따라, 본 과정에서는 annotation 수가 400 개 이상인 알약 클래스를 outlier 로 정의하고, 해당 클래스의 데이터 수를 300 개로 제한하는 방식으로 조정하였다

```
=== annotation 수 구간별 카테고리 개수 ===
1~100      : 62 cat
101~200    : 9 cat
201~300    : 1 cat
301~400    : 0 cat
400+       : 1 cat
```

그림 6. cat_id 수에 따른 cat_id 그룹화

- Weighted Random Sampler: 소수 클래스에 높은 가중치를 부여하여 균형 있는 학습 유도
 - B. Weighted Random Sampling 방식은 데이터 불균형 문제를 완화하기 위한 방법 중 하나로, 데이터 수가 적은 클래스에 더 높은 가중치를 부여하여 학습 과정에서 해당 클래스의 샘플이 더 자주 선택되도록 하는 원리이다. 본 과정에서는 데이터 불균형 문제를 해결하기 위해 학습 과정에서 sampling 을 적용하였으며, 이를 Weighted Random Sampler 로 추가하여 학습시켰다.
- Image Augmentation: HorizontalFlip, PhotometricDistort, RandomRotation 등 조합 실험

- C. 데이터셋의 학습성능 향상을 위해, 이미지 증강기법을 적용하였다. 크게 형태 변환과 조명 변화에 초점을 맞추었다. 형태 변환의 경우, 이미지의 구조적 변화를 반영하고자 flip 과 rotation 을 사용하였으며, 색조의 경우, 알약과 배경 사이의 색상 대비 및 채도에 변주를 주하고자 photometric 변환을 적용하였다. 이 과정에서, 과도한 변형을 방지하고자 RandomHorizontalFlip(p=0.3), RandomPhotometricDistort(p=0.3), 그리고 RandomRotation(degrees=15)와 같이 그 확률을 낮게 제한하였다. (그림 7 참조)

```
train_transforms = v2.Compose([v2.ToImage(),
                                v2.RandomHorizontalFlip(p=0.3),
                                v2.RandomPhotometricDistort(p=0.3),
                                #v2.RandomRotation(degrees=15),
                                v2.ToDtype(dtype=torch.float32, scale=True),
                                v2.Normalize(mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225))
                                ])
```

그림 7. Img Augmentation

- Model(Faster R-CNN)성능 평가:

- D. 앞서 수행한 데이터 전처리 방법의 효과를 확인하기 위해, Baseline 으로 제공된 Faster R-CNN 모델에 해당 전처리 기법들을 적용하여 성능 평가를 진행하였다. 실험에서는 augmentation 적용 방식을 두가지 설정으로 나누었다. Conditional Augmentation 은 데이터 수가 적은 클래스에 한해 조건적으로 증강을 적용하는 방식이며, All Augmentation 은 전체 train dataset 에 대해 동일하게 증강을 적용한다. 또한, 실험 과정에서 다양한 augmentation 조합을 비교하기 위해 일부 증강 기법을 추가하거나 제거하고, 적용 확률을 조정하는 등의 방식으로 수행하였다. 그 결과, epoch 등 학습 조건에서 차이가 있긴하나, 표 1 에서 보는 바와 같이, Weighted Sampling 기법만을 적용한 경우 가장 안정적인 성능을 보였다.

특히, sampler 는 데이터 수가 적은 클래스에 더 높은 가중치를 부여하는 방식으로, 상대적으로 데이터 수가 많은 클래스의 영향력을 줄이는 효과가 있기에, 결과적으로 outlier handling 과 유사한 역할을 수행한다고 판단하였다. 이에 따라 불필요한 연산을 줄이고자 outlier 처리과정은 제거하는 방향으로 진행하였다.

또한, augmentation 의 경우 여러 조합을 실험한 결과, 최종적으로 아래와 같은 augmentation 구성을 적용하였다.

1. [epoch=5] Outlier handling + Sampler (=> 0.88)

```
[Epoch 5/5] train_loss: 0.3635 | val_loss: 0.0970 | mAP@50: 0.8879 | mAP@50:95: 0.8548
```

2. [epoch=5] Outlier handling + Sampler + conditional Aug (HorizontalFlip(p=0.3),PhotometricDistort(p=0.5) (=> 0.86)

```
[Epoch 5/5] train_loss: 0.4099 | val_loss: 0.1170 | mAP@50: 0.8632 | mAP@50:95: 0.8123
```

3. [epoch=5] Outlier handling + Sampler + all Aug(HorizontalFlip(p=0.3),PhotometricDistort(p=0.3) (=> 0.81)

```
[Epoch 5/5] train_loss: 0.1223 | val_loss: 0.9901 | mAP@50: 0.8142 | mAP@50:95: 0.7680
```

4. [epoch=10] Sampler (=> 0.89)

```
[Epoch 10/10] train_loss: 0.0973 | val_loss: 0.0706 | mAP@50: 0.8976 | mAP@50:95: 0.8708
```

5. [epoch=10] Sampler + conditional Aug (HorizontalFlip(p=0.3),PhotometricDistort(p=0.3) (=> 0.88)

```
[Epoch 10/10] train_loss: 0.2428 | val_loss: 0.0797 | mAP@50: 0.8884 | mAP@50:95: 0.8670
```

6. [epoch=10] Sampler + all Aug (HorizontalFlip(p=0.3),PhotometricDistort(p=0.3)(=> 0.87)

```
[Epoch 10/10] train_loss: 0.3967 | val_loss: 0.0871 | mAP@50: 0.8759 | mAP@50:95: 0.8482
```

표 1. 조건에 따른 성능결과

```
train_transforms = A.Compose([A.Resize(640, 640),
                              A.ShiftScaleRotate(shift_limit=0.1, scale_limit=0.2, rotate_limit=30, border_mode=0, p=0.8),
                              A.HorizontalFlip(p=0.5),
                              A.VerticalFlip(p=0.5),
                              A.RandomRotate90(p=0.5),
                              A.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1, p=0.5),
                              A.CLAHE(clip_limit=4.0, p=0.4),
                              A.GaussNoise(p=0.3),
                              A.MotionBlur(blur_limit=3, p=0.2),
                              A.Normalize(mean=(0.485, 0.456, 0.406),std=(0.229, 0.224, 0.225)),
                              ToTensorV2()),
                              bbox_params=A.BboxParams(format = 'coco',
                                                         label_fields = ['category_ids'],
                                                         min_area = 1.0,
                                                         min_visibility = 0.1,
                                                         clip = True )) # bbox 이탈문제로 clip해줘야함.
```

그림 8. 추가 Augmentation

실험 조건	epoch	mAP@0.5
Outlier + Sampler	5	0.88
Outlier + Sampler + Conditional Aug	5	0.86
Outlier + Sampler + All Aug	5	0.81
Sampler만 적용	10	0.89 (최고)
Sampler + Conditional Aug	10	0.88
Sampler + All Aug	10	0.87

실험 결과, Weighted Sampler만 단독 적용한 경우가 가장 안정적인 성능을 보였다. Augmentation 추가 시 오히려 성능이 소폭 하락하는 경향이 관찰되었으며, 이는 Sampler가 Outlier Handling과 유사한 역할을 수행하기 때문으로 판단하였다.

이후 AMP(Automatic Mixed Precision) 및 Gradient Accumulation 적용으로 메모리를 최적화 하고(batch_size 16, 이미지 800×800), epoch·scheduler 조합 실험을 진행하였다. StepLR(step_size=7, gamma=0.5) 기준 epoch 30에서 mAP@0.5 최고 0.904를 달성하였다.

추론 단계에서 Normalize 누락이 발견되었으며, 이를 수정하자 Kaggle 점수가 0.675 → 0.816 으로 0.14 이상 향상되었다. 학습·추론 전처리 일관성의 중요성을 직접 확인한 사례이다.

** Model Architecture

Faster R-CNN 모델 외에 Pills Detection에 적합한 모델을 탐색하고자 여러 모델의 특징과 적용 가능성을 비교 분석하였다.

모델	장점	적합성	ref.
Mask R-CNN	pixel로 학습하여, 객체의 segmentation mask를 예측할 수 있는 모델로, overlapping img에서 객체를 구분하는 데 유리함.	제공된 data img는 서로 분리되어 각 영역이 나누어져있는 구조로, pixel로 학습시키는 방식이 연산만 증가시킬 뿐, 주어진 데이터를 detection하는데 부적합하다고 판단함.	3)
+ inception V3	이미지 classification성능은 좋은 편임.	Inception v3.에 대해서 평가 결과가 나쁜 편은 아니나, 단독으로 사용하기에 Pill Detection에 부적절하므로, 사용하면, faster R-CNN에 +inceptionv3을 결합한 모델로 제안.	1), 2)
Yolo v.5	가벼운(lightweight) 모델로 실시간 처리에 적합함.	실시간 환경을 사용하는 App의 경우 (경량성+정확도) 유리할 수 있지만, 본 프로젝트는 모델성능, 정확도 중심으로 YOLO모델의 우선순위에서 제외함.	6)
Yolo v.8s/8n	Pill Detection에서 가장 많이 활용되는 모델	알약 검출 분야에서 활용 사례가 많아 적합한 모델로 판단하여 적용함.	4) 6)
Yolo v.11	v8 이 가장 흔히 사용되는 추세이나, 최신 버전인 v11을 활용한 pill detection연구들도 존재.	최신 연구에서 활용 사례가 보고되고 있어, 실험해 볼 가치가 있다고 판단함.	5), 7)

표2. 모델 적합성 분석

3.2 Phase 2 — Backbone 교체 실험 (Faster R-CNN)

더 나은 성능을 위해 Faster R-CNN의 Backbone을 ResNet50에서 교체하는 실험을 진행하였으나 모두 성능 저하로 채택하지 않았다.

Backbone	Kaggle mAP	비고
ResNet50 (기준)	0.975	증강+샘플러 적용 기준
ResNet101	0.117~0.264	OOM, 학습 미흡
ResNet34	0.359	기준 대비 낮음

실험 결과, 데이터 규모가 작은 경우 복잡한 모델이 과적합되거나 학습이 불안정해질 수 있음을 확인하였다. 이후 팀 전원 동의 하에 YOLO 계열로 전환하기로 결정하였다.

3.3 Phase 3 — YOLO 모델 도입 및 디버깅

**** YOLO model v8 & YOLO model v11**

YOLOv8과 YOLOv11은 그림 9와 10에서 보는 바와 같이, backbone과 neck 구조에서 차이가 존재하나, 큰 차이는 backbone에서 사용되는 모듈 구조에 있다고 할 수 있다. YOLOv8은 C2f(Cross Stage Partial with feature fusion) 구조를 기반으로 특징 추출을 수행하는 반면, YOLOv11은 C3K2 구조를 중심으로 설계되어 있으며 일부 블록에 PSCA(Partial Self-Calibrated Attention)메커니즘이 적용되어 있다. 이 Attention 구조는 feature map 내에서 중요한 정보를 강조하여 객체 표현력을 향상시키는 것을 목적으로 한다.

이러한 구조적 차이를 바탕으로, 실험에 앞서 Attention 메커니즘이 포함된 YOLOv11이 더 높은 성능을 보일 것이라 추측하였다.

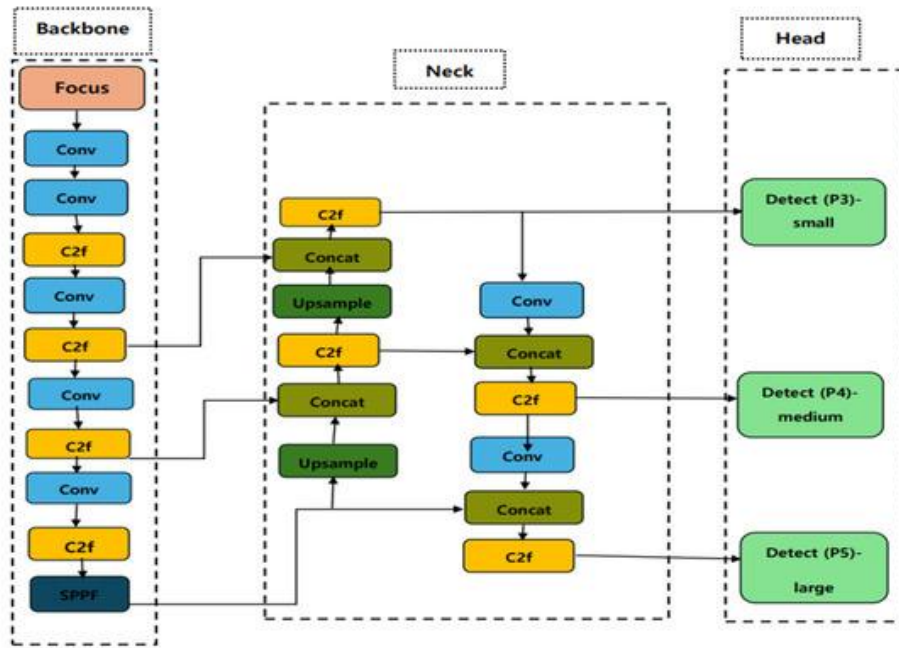


그림 9. YOLO v8 model

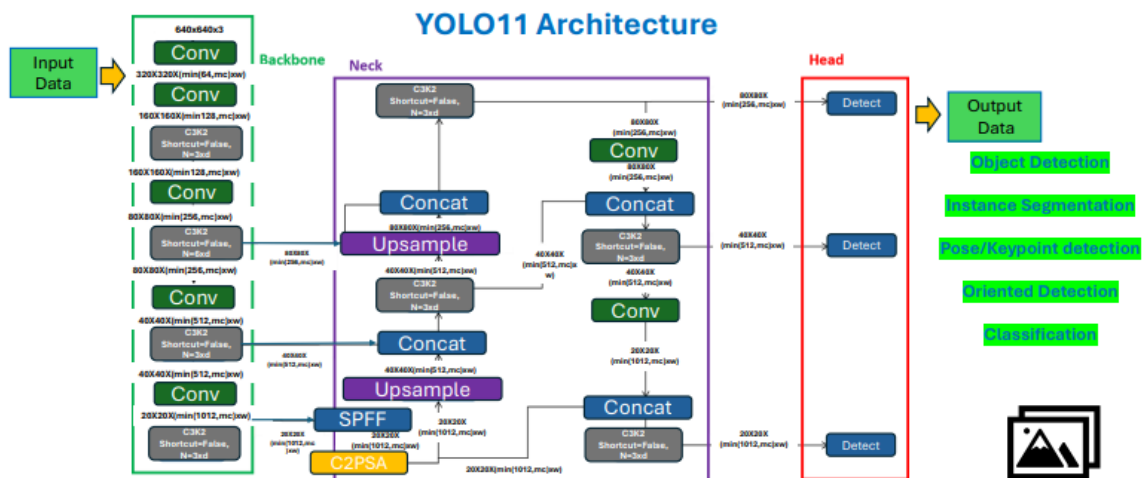


그림 10. YOLO v11 model

YOLO 계열로 전환 후 팀원들이 YOLOv8, YOLOv11 기반 실험을 병행하였으나, 초기에는 Kaggle 제출 시 0점이 반복 발생하였다. 주요 원인은 다음과 같았다.

- category_id 매핑 오류: Faster R-CNN 은 0 번을 배경으로 예약(1-based)하지만, YOLO 는 0-based 방식을 사용함. +1 보정 코드가 잘못 적용된 것이 원인

- 데이터셋 분할 방식 오류: train/test 데이터를 잘못 사용하여 mAP 가 0.35 수준으로 매우 낮게 나옴
- label encoding 문제: YOLO 학습 단계의 라벨링 코드 오류로 인한 0 점 발생

이러한 문제들을 팀원 간 코드 비교 및 디버깅을 통해 순차적으로 해결하는 과정에서, 팀장(유연정)이 YOLOv8s 코드를 직접 작성하여 실험하였다. 첫 제출에서 Kaggle 0.989를 기록하며 기존 실험 대비 최고 성능을 달성하였다.

시도	Kaggle mAP	비고
YOLOv8s 최초 제출	0.989	팀 최고점 달성
imgsz 1280 적용 후	0.993	이미지 사이즈 조정
YOLOv11s 증강 없이	0.992	8s 대비 낮음
YOLOv8s 증강 적용	0.992	증강이 오히려 성능 저하
YOLOv8s 최종	0.994	최종 채택

증강 유/무 비교 실험에서, 알약처럼 배경이 단순하고 객체 형태가 일정한 데이터에서는 과도한 증강이 학습을 방해할 수 있음이 확인되었다. 이후 증강을 배제하고 하이퍼파라미터 튜닝에 집중하는 방향으로 결정하였다.

3.4 Phase 4 — 모델 비교 실험 및 하이퍼파라미터 튜닝

YOLOv8s를 기준 모델로 채택한 후, 더 나은 모델이 있는지 확인하기 위한 비교 실험을 수행하였다.

비교 모델	Kaggle mAP	비고
YOLOv8s (기준)	0.99451	최종 채택
YOLOv8m (epoch 50)	0.98833	기준 대비 낮음
YOLOv11n	0.97910	기준 대비 낮음
YOLOv11s	0.992	기준 대비 낮음
YOLOv11n + 모자이크 증강	최대 0.83	기준 대비 낮음

YOLOv8s 모델의 성능을 추가로 끌어올리기 위한 초정밀 하이퍼파라미터 튜닝 실험을 다수 진행하였다. 튜닝 방향은 크게 세 가지로 나누어 접근하였다.

첫째, 기하학적 강건성 확보를 위한 Augmentation 전략이다. degrees_translate 파라미터를 조정하여 알약의 임의 회전 및 위치 변화에 대응하는 시각적 변수를 추가하였고, mixup_overlap_mask를 적용하여 알약이 겹치거나 밀집된 환경에서의 탐지 정확도 개선을 시도하였다.

둘째, 학습 안정화 및 최적화 전략이다. Cosine LR(cos_lr)을 적용하여 학습률을 부드럽게 감소시켜 최적 수렴 지점(Global Minimum) 안착을 유도하였고, AdamW 옵티마이저로 가중치 업데이트 효율성을 높였다. Label Smoothing(0.05~0.1)은 모델이 과도하게 확신하는 것을 방지하여 일반화 성능 향상을 목적으로 적용하였다.

셋째, 최종 정밀도 마감을 위한 Fine-tuning이다. Close Mosaic 옵션으로 학습 종료 전 마지막 몇 에폭에서 모자이크 증강을 끄고 원본 데이터에 대한 정밀 복습을 유도하였으며, TTA(Test Time Augmentation)를 추론 단계에 적용하여 다각도 검토를 통한 결과 신뢰도를 높이고자 하였다.

이러한 일련의 조합 실험을 5회 이상 반복하였으나, 최고 점수인 0.99451 이상의 추가 개선은 이루어지지 않았다. Optuna를 활용한 자동 하이퍼파라미터 탐색도 시도하였으나, Colab 세션 중단(백그라운드 연결 끊김, 인터넷 단절 등)으로 최종 결과를 얻지 못하였다.

4. 팀원별 기여

4.1 유연정 (PM / 데이터 전처리 / 모델 설계 및 실험)

팀 전체 일정 관리 및 방향 조율을 수행하는 PM 역할을 담당하는 동시에, 데이터 전처리와 모델 실험에 직접 참여하였다. 팀원들이 공통적으로 겪던 오류의 원인을 찾아 공유하고, 최종 채택 모델 코드를 직접 작성하는 등 실험 전반에 걸쳐 핵심적인 기여를 하였다.

주요 기여 내용은 다음과 같다.

- 데이터 분석 및 시각화: 데이터 불균형 확인, YOLO 전용 데이터셋 클래스 작성, 라벨 정합성 검증
- 코드 리팩토링: torchvision v2 transforms 로 교체하여 bbox 와 이미지 동시 변환 구현, train/val transforms 분리로 평가 왜곡 방지
- wandb 파이프라인 구축: 실험별 run_name 자동 분리 구조 설계, 팀 전원이 동일한 대시보드에서 결과 비교할 수 있는 기반 마련
- 메모리 최적화: AMP(torch.amp.autocast) + Gradient Accumulation 적용, batch OOM 원인(batch_size=32) 파악 및 수정
- 추론 오류 발견: 추론 시 Normalize 누락 확인 및 수정 → Kaggle 점수 0.675 → 0.816 으로 0.14 향상
- Backbone 교체 실험: ResNet50/101/34 비교 실험 수행, 팀 의사결정 근거 마련
- YOLO 전환 주도: category_id 매핑 오류(FRCNN 1-based vs YOLO 0-based) 원인 파악 및 팀 공유
- 최종 모델 작성: YOLOv8s 코드 직접 작성 및 검증, 0.989 → 0.993 → 0.994 점진적 향상
- 팀원 배포용 노트북 완성 및 검증, 전처리 일관성 기준 수립

4.2 김현숙 (데이터 전처리 / 모델 설계 및 실험)

데이터 분석·전처리 전략 수립부터 모델 아키텍처 비교까지 프로젝트 전 구간에 걸쳐 실질적인 기여를 하였다. 팀 전체가 사용하는 데이터 파이프라인의 기준을 설계하고 코드를 정비하는 역할을 담당하였으며, 논문 기반의 체계적인 모델 분석으로 팀의 모델 선택 의사결정에 근거를 제공하였다.

- EDA: 이미지 1,489 장·annotation 4,526 개·73 카테고리 구조 분석; 클래스 불균형(최대 514 개 vs 최소 7 개)과 annotation 누락 문제를 수치로 규명
- 전처리 전략 설계: Outlier Handling·Weighted Random Sampler·Image Augmentation 세 가지 방향을 제안하고 6 가지 조합 실험을 통해 'Sampler 단독 적용'이 가장 안정적임을 실험으로 도출
- 데이터 정규화·cat_index 수정·불필요한 Outlier Handling 제거 후 팀 GitHub Pull Request 로 기준 코드 배포
- 모델 적합성 비교 분석: 알약 탐지 관련 논문 8 편을 직접 조사하여 Mask R-CNN·Inception V3·YOLO v5/v8/v11 각각의 적합성과 한계를 정리, 참고문헌 목록 작성
- YOLOv8 vs YOLOv11 내부 구조 비교: C2f(v8)와 C3K2+PSCA 메커니즘(v11)의 차이를 분석하여 데이터 특성 기반으로 v8 이 더 적합한 이유를 이론적으로 뒷받침
- JSON → YAML 데이터 변환 코드 작성, YOLO 학습용 데이터셋 구성
- YOLOv8s Transfer Learning 및 EfficientNet 결합 실험 시도

4.3 이현진 (모델 설계 및 실험 / PPT 작성)

모델 아키텍처 탐색과 발표 자료 제작을 담당하였다. 프로젝트 초기부터 YOLOv8을 1순위로 제안하였으며, 모델 선택 기준을 최신성보다 적용 난이도·코드 변경량·학습 안정성·데이터 규모 적합성에 두고 후보를 체계적으로 비교하였다.

- 모델 우선순위 제안: 허깅페이스 및 외부 자료를 직접 조사하여 YOLOv8 (1 순위)·Faster R-CNN ResNet50 FPN v2 (2 순위)·YOLOv8+TTA/Ensemble (3 순위)으로 정리; DETR 계열은 데이터 요구량 과다로 제외 — 팀 최종 선택과 방향 일치
- wandb commError 원인 파악: 개인 entity 불일치 문제를 발견하고 팀 공용 entity 로 변경 제안, 팀 전체 협업 리스크를 사전에 줄이는 데 기여
- YOLOv8 직접 적용 및 TTA 실험: model.val()로 mAP@0.5·mAP@0.5:0.95 확인; 초기 학습 단계에 TTA 를 삽입하여 오류가 발생하였으나 추론 단계(predict)에만 적용하도록 수정
- YOLOv11 실험 시도 중 에러 발생, 위치 수정과 코드 재정비를 통해 재실행 방향을 잡았음
- 발표 자료(PPT) 작성 담당: Genspark 를 활용하여 팀 발표 자료 제작, 최종 정리 단계에서 실험 마무리와 병행

4.4 송우현 (모델 설계 및 실험)

다양한 모델과 하이퍼파라미터 조합을 적극적으로 시도하며 팀의 실험 데이터를 축적하였다. 결과가 좋지 않더라도 포기하지 않고 원인을 분석하며 반복 실험을 이어갔으며, 여러 모델의 비교 수치를 직접 산출하여 팀이 최종 모델을 선정하는 데 필요한 근거를 마련하는 데 기여하였다.

- 초기 전처리 보강: RandomRotation(0~360 도), GaussianBlur(kernel_size=5~9)를 추가하여 알약이 어떤 방향으로 놓여도 인식 가능하고 화질이 낮은 실전 사진에도 강건하게 대응할 수 있도록 증강 전략을 제안하였음
- Optuna 도입: Faster R-CNN 대상으로 lr·weight_decay·batch_size·optimizer 를 자동 탐색하는 실험을 설계하고 수행; 탐색 과정에서 배치 크기를 상향 조정하던 중 bbox 좌표가 이상한(면적이 0 인) 데이터를 발견하고 학습 루프 내 필터링 코드를 삽입하여 해결
- YOLO 0 점 문제 디버깅: 팀원들이 YOLO 모델을 Kaggle 에 제출할 때 반복적으로 0 점이 발생하는 원인을 label encoding 및 category_id 매핑 오류로 파악하고, 팀원과 코드를 직접 비교·대조하며 공동으로 해결
- YOLOv11n (0.97910)·YOLOv8m (0.98833) 비교 실험을 직접 수행하여 현재 데이터셋 조건에서 YOLOv8s 가 가장 적합한 모델임을 수치로 입증, 팀의 최종 모델 확정에 근거를 제공
- 하이퍼파라미터 미세조정 5 회 이상 시도: Cosine LR·Label Smoothing·Close Mosaic·AdamW·degrees·mixup 등 다양한 조합을 실험하며 최고점 이상의 성능 향상 가능성을 끝까지 탐색하였으나 0.99451 을 넘지 못하였음

4.5 은광성 (데이터 전처리 / 모델 실험)

데이터 전처리와 모델 증강 실험을 담당하였다. 특히 Albumentation 라이브러리를 활용한 모자이크 증강 기법을 팀 내에서 주도적으로 탐색하고 적용하였으며, 실험 결과를 팀에 공유하여 전처리 전략 결정에 기여하였다.

- 베이스라인 코드를 직접 실행하고 시각화하여 데이터 구조와 학습 흐름을 파악하는 초기 탐색 작업 수행
- Albumentation 기반 모자이크 증강 코드 설계 및 적용:
ShiftScaleRotate·HorizontalFlip·VerticalFlip·ColorJitter·CLAHE·GaussNoise·MotionBlur 조합으로 다양한 촬영 환경 변수를 학습에 반영

- 10 epoch 에서 mAP 0.89, 30 epoch 에서 mAP 0.912 달성, 결과를 팀에 공유하여 팀 전체리 전략 논의에 반영됨
- YOLO 전환 후 YOLOv11n + 모자이크 증강 결합 실험 시도(최대 mAP 0.83), epoch·배치·학습률 등 조건을 변경하며 반복 실험 수행
- wandb 연동 및 모니터링 환경 구성; CSV 추출·mAP 도출 과정의 반복 오류를 파악하고 파일 재작성으로 끝까지 해결을 시도

5. 주요 이슈 및 해결

이슈	원인	해결 방법
Kaggle 0점 반복 발생	YOLO 0-based vs FRCNN 1-based category_id 매핑 혼동	orig2model 매핑 수정, 추론 셀의 +1 보정 제거
추론 점수 급락	추론 시 Normalize 누락으로 학습 /추론 전처리 불일치	T.Normalize 추가 후 0.675 → 0.816 (+0.14)
CUDA OOM 에러	batch_size=32, 이미지 1280×1280 조합	AMP 적용 + batch_size 16 + Resize(800,800)
Backbone 교체 성능 저하	소규모 데이터에 ResNet101/34 과적합 또는 학습 부족	YOLO 계열로 전환
Optuna 결과 유실	Colab 백그라운드 세션 자동 종료	50 trials → 30 → 10으로 축소 시도, 최종 포기
TTA 적용 위치 오류	학습 단계에 TTA 삽입으로 오류	추론 단계(predict)에만 augment=True 적용
wandb commError	개인 계정 entity 불일치	팀 공용 entity 기준으로 변경
증강 적용 후 성능 저하	단순·균일한 데이터에 과도한 증강이 학습 방해	증강 제거 후 순정 YOLOv8s 채택
StepLR 과도한 lr 감소	step_size=3, gamma=0.1로 epoch 9 이후 lr이 1e-7로 수렴	step_size=7, gamma=0.5로 완화

6. 파일 구조

프로젝트 주요 파일 구성은 다음과 같다.

Oral_Pill_Image_Recognition_Team3/

├── Project_1_Team3_Final.ipynb # 최종 제출 코드 (YOLOv8s 기반)

├── Team_3_Pill-터링_Report.pdf # 팀 보고서

├── Team_3_Pill-터링_Presentation.pptx # 발표 자료

└── README.md # 데이터셋 경로 및 환경 설정 안내 포함

 # (학습 데이터는 Google Drive에서 별도 다운로드)

회의록, 팀원 협업 노트, 프로젝트 가이드라인 및 일정표 등 팀 협업 기록은 README에 명시된 Notion 페이지에서 확인할 수 있다.

7. 결론 및 회고

본 프로젝트는 경구용 알약 이미지 분류를 위한 객체 탐지 모델 개발 프로젝트로, 최종적으로 Kaggle mAP 0.99451을 달성하여 1위를 기록하였다.

실험 과정에서 얻은 주요 인사이트는 다음과 같다.

- 소규모 특화 데이터셋에서는 복잡한 모델(ResNet101, YOLOv11, DETR 계열)보다 경량 모델(YOLOv8s)이 더 효율적이고 안정적인 성능을 보였다.
- 데이터 불균형 완화에는 Outlier Handling 이나 Augmentation 보다 Weighted Random Sampler 가 더 효과적이었다.
- 증강은 데이터 특성에 맞는 적절한 종류와 강도가 중요하다. 배경이 단순하고 객체 형태가 일정한 알약 이미지에서는 과도한 증강이 오히려 성능 저하를 유발하였다.
- 학습과 추론 단계의 전처리 일관성이 점수에 결정적인 영향을 미쳤다. Normalize 하나의 누락만으로도 0.14 이상의 점수 차이가 발생하였다.
- TTA 는 추론 단계에서만 적용해야 하며, category_id 매핑 체계(FRCNN 1-based vs YOLO 0-based)를 정확히 이해하는 것이 Kaggle 제출 오류를 방지하는 핵심이었다.

팀원 모두가 각자 맡은 역할에서 실험과 디버깅을 성실히 수행하였으며, 매일 스크럼 회의와 협업 노트를 통한 지속적인 소통이 팀 전체의 실험 효율을 높이는 데 크게 기여하였다.

8. 참고자료

■ 알약 탐지 관련 논문

- 1) My, L.N.T., Le, V., Vo, T., Hoang, V.T. (2025). A Comprehensive Review of Pill Image Recognition. *Computers, Materials & Continua*, 82(3), 3693–3740. <https://doi.org/10.32604/cmc.2025.060793>
- 2) Larios Delgado, N., Usuyama, N., Hall, A. K., et al. (2019). Fast and accurate medication identification. *npj Digital Medicine*, 2, 10. <https://doi.org/10.1038/s41746-019-0086-0>
- 3) Kwon, H.-J., Kim, H.-G., & Lee, S.-H. (2022). Pill detection model for medicine inspection based on deep learning. *Chemosensors*, 10(1), 4. <https://doi.org/10.3390/chemosensors10010004>
- 4) Dang, B. et al. (2024). Real-time pill identification for the visually impaired using deep learning. *CISCE 2024*. <https://doi.org/10.48550/arXiv.2405.05983>
- 5) Kim, S. J., & Cho, D. S. (2025). Medical-pills detection using YOLOv11. *Research Square* (Preprint v2). <https://doi.org/10.21203/rs.3.rs-6337589/v2>

■ YOLO 계열 모델

- 6) Ali, M. L., & Zhang, Z. (2024). The YOLO framework: A comprehensive review of evolution, applications, and benchmarks in object detection. *Computers*, 13(12), 336. <https://doi.org/10.3390/computers13120336>
- 7) Sapkota, R., & Karkee, M. (2025). Ultralytics YOLO evolution: An overview of YOLO26, YOLO11, YOLOv8 and YOLOv5 object detectors for computer vision. *arXiv:2510.09653*. <https://arxiv.org/abs/2510.09653>
- 8) Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. *CVPR 2016*. <https://doi.org/10.1109/CVPR.2016.91>
- 9) Ultralytics YOLO Documentation. <https://docs.ultralytics.com/models/>

■ Object Detection 기반 모델

- 10) Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. *NeurIPS 2015*. <https://doi.org/10.48550/arXiv.1506.01497>
- 11) He, K., Gkioxari, G., Dollár, P., & Girshick, R. (2017). Mask R-CNN. *ICCV 2017*. <https://doi.org/10.48550/arXiv.1703.06870>
- 12) Carion, N., Massa, F., Synnaeve, G., et al. (2020). End-to-End Object Detection with Transformers (DETR). *ECCV 2020*. <https://doi.org/10.48550/arXiv.2005.12872>

■ 데이터 증강 및 학습 기법

- 13) Buslaev, A., Iglovikov, V. I., Khvedchenya, E., et al. (2020). Albumentations: Fast and Flexible Image Augmentations. *Information*, 11(2), 125. <https://doi.org/10.3390/info11020125>
- 14) Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on Image Data Augmentation for Deep Learning. *Journal of Big Data*, 6, 60. <https://doi.org/10.1186/s40537-019-0197-0>
- 15) Akiba, T., Sano, S., Yanase, T., et al. (2019). Optuna: A Next-generation

Hyperparameter Optimization Framework. KDD 2019.
<https://doi.org/10.48550/arXiv.1907.10902>

■ 참고 블로그 및 문서

16) Medical-pills detection using Ultralytics YOLO11. PixelMindX.
<https://medium.com/pixelmindx/medical-pills-detection-using-ultralytics-yolo11-d36d8429a6c6>

17) Weights & Biases Documentation. <https://docs.wandb.ai/>

18) PyTorch torchvision v2 Transforms Documentation.
<https://pytorch.org/vision/stable/transforms.html>